

网络篇-2

一、BGP详解

第一、基本概念

在IGP中，设备与设备之间在底层网络上互通的，当AS边界路由器从外界学到了10万条路由，如果塞进OSPF，会造成CPU宕机，因为OSPF里面会存拓扑信息，而BGP是点对点，它不会泛洪，存的也只是路由信息，不会造成宕机

AS_Path可以解决BGP区域间防环问题，而水平分割可以解决AS区域内的防环问题，但是水平分割又会造成设备之间无法连通的问题，于是就出现了全互联技术，A 想要让 C 知道这 10 万条路由，唯一的办法就是：**A 越过 B，直接跟 C 建立 IBGP 邻居关系**，也就是A直接和C建立TCP连接并建立IBGP关系，那么B无需关心A发来的是什么，直接做转发处理就行

在规模达到一定程度之后，即使是全互联，任然会造成很大负载，于是又出现了路由反射器和联邦的概念

RR：路由反射器：

核心思想非常霸道：直接把水平分割废掉！它在网络中设立一个“群主”（也就是 RR），其他的路由器变成“群成员”（Client）。群成员之间不需要互相建立 TCP 连接，所有人都只和“群主”建邻居。

为了让这套机制完美运转，RR 体系定义了三种角色和极其严格的“**反射规则**”：

RR (路由反射器)：群主，负责把收到的路由“反射”给其他人

Client (客户)：群成员，它们完全不知道 RR 的存在，在它们眼里，RR 就是一个普通的 IBGP 邻居

Non-Client (非客户)：不在这个群里、但也和 RR 建立了 IBGP 关系的普通路由器

面试必考点：当 RR 收到一条路由时，它会怎么转发？

从客户收到的路由：反射给所有的 客户和非客户。（群成员发的消息，群主广播给群成员）

从非客户收到的路由：只反射给客户，绝不反射给其他非客户

从 EBGP 收到的路由：传给所有的 客户和非客户

既然 RR 胆敢打破 IBGP 的防环神针（水平分割），那它万一把路由反射了一圈又发回给自己，形成死环怎么办？

Originator_ID (起源者 ID)：当 RR 反射一条路由时，会把最开始发出这条路由的 Client 的 Router ID 记录在里面。如果这台 Client 后来又收到了这条路由，一看 Originator_ID 是自己，立刻丢弃。

Cluster_List (集群列表)：如果网络很大，可能会有多个 RR（多个群）。当路由每经过一个 RR 时，RR 就会把自己的 Cluster_ID（通常是它的 Router ID）追加到这个列表里。如果 RR 收到一条路由，发现 Cluster_List 里面已经有自己的名字了，立刻丢弃。（我发现这部分很类似AS_set的机制）

联邦

我的理解就是大AS里面套小AS，这样水平分割在小AS里面就不受限制了，可以在不实现全互联的情况下，解决路由传递问题

第二、路由属性

说路由属性之前，先看看路由汇总

在 BGP 中，AS_Path 是用来防环的（记录了路由经过的所有 AS 号）。但是，当你把多条来自不同 AS 的明细路由汇总成一条大路由时，原来的明细 AS_Path 细节就丢失了！汇总路由器默认只会给这条汇总路由打上自己的 AS 号。要解决这个问题，可以使用

Atomic_Aggregate：这是一个警告标志。当路由器发送一条汇总路由，并且隐藏了明细路由时，必须带上这个属性。它是在告诉下游的所有路由器：“注意！这是一条被浓缩过的路由，里面原本带的一些具体路径信息被我抹掉了，你们用的时候要小心环路盲区！”

Aggregator：记录了是在哪个 AS 里、被哪台路由器（Router ID）执行了这次汇总操作，方便工程师顺藤摸瓜去排错

AS-set：如果加上 `as-set` 参数，BGP 会把所有明细路由里出现过的 AS 号，打包塞进一个无序的集合 `{}` 里，作为汇总路由的 AS_Path 发出去。

Local_Pref

只在AS内部传递，越大越优。谁的值大，整个 AS 内部的流量就往谁那里走，也就是该属性可以被AS用于告知AS内的BGP路由器哪一个出口优先级更高

AS_Path

一台路由器在将BGP路由通告给自己的EBGP对等体时，将本地的AS号插入到该路由原有的AS_Path之前。它可以用来实现EBGP路由的环路避免，也可以用于BGP路由优选的决策

Origin

主要是描述BGP路由的来源

- **IGP**：如果路由是由始发的 BGP 路由器使用 `network` 命令发布到 BGP 的，那么该 BGP 路由的 Origin 属性为 IGP。
- **EGP**：如果路由是通过 EGP 学习到的，那么该 BGP 路由的 Origin 属性为 EGP。



此处 EGP 指的是一个具体的路由协议，其名称为 EGP。这个路由协议现在已经不再被使用。

- **Incomplete**：如果路由是通过其他方式学习到的，则 Origin 属性为 Incomplete（不完整的）。例如通过 `import-route` 命令引入到 BGP 的路由。

MED

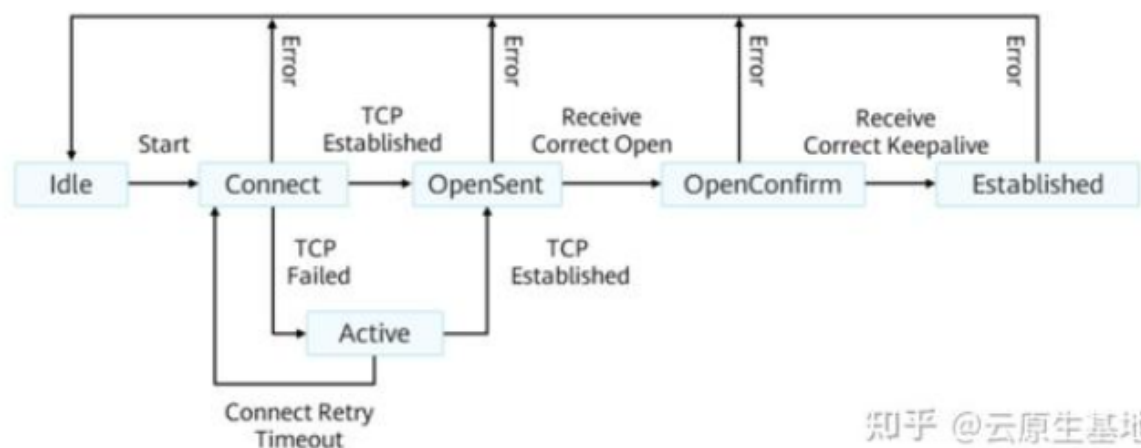
相较于local_preference控制本AS的流量从哪个入口出去，MED用于控制外部AS的流量从哪一个入口进来，但注意是面向同一个对象，值越小越优

next_Hop

一台路由器将 BGP 路由发送给 EBGP 邻居（跨 AS）时，它会自动把 Next_Hop 属性修改为自己的发送接口 IP 地址。

当一台路由器将从 EBGP 学到的路由，再次转发给内部的 IBGP 邻居时，默认情况下，它不会修改 Next_Hop 的值。所以，如果 Next_Hop 在底层的路由表里连通不了，这条 BGP 路由就会被视为“无效路由”，根本进不了转发表。一旦业务数据包发过来，因为查不到有效表项，就会被直接丢弃，导致严重的网络不通。不过好在可以使用命令修改属性值

第三、状态机



Idle: 空闲状态

路由器刚刚配置了BGP进程，或者刚刚经历了一次严重的报错，比如Notification，如果一直是这个状态，可能是因为去往对端的IGP没通，或者端口被强制shutdown了

connect: 连接状态

在这个状态，路由器主动向邻居发起TCP三次握手，如果底层路由是通的，且TCP握手成功，它会立刻向对方发送 `Open` 报文，并跳转到 **OpenSent** 状态，如果TCP握手超时（比如包丢了，或者对端没响应），它就会跳转到 **Active** 状态

active: 活跃状态

是一个不健康的状态。它意味着第一次TCP握手失败了，如果状态一直在Connect和Active之间反复横跳，说明路由器正在急躁地不断重试建立TCP连接。可能是防火墙拦截，对方拒绝TCP连接请求

opensent: 审查阶段

TCP三次握手终于成功了！双方建立了可靠的传输通道。此时，路由器向对端发送了 `Open` 报文，当收到对端的open报文后，路由器会进行审查，如果发现对端发来的AS号和自己本地配置的不一样，或者Router ID冲突了。它会发送一个 `Notification` 报错报文，并直接退回到Idle状态

openconfirm: 审查通过

这个状态下，意味着审查完了对方发来open报文的参数，路由器发出了 `keepalive` 报文。它现在唯一要做的，就是等待对方也发来一个 `keepalive` 报文

Established: 建立状态

到了这个状态，说明TCP通了，参数对上了，邻居关系正式确立。此时，双方就可以开始疯狂地发送 `update` 报文交换那几十万条路由表了，在Established状态下，双方会周期性地（通常是60秒）互发 `keepalive` 报文，确认对方还活着

第四、总结

支持自动、手动路由汇总，TCP179，触发式更新，建立对等体关系无需直连

AS_path的长度会影响路由器对BGP路径的优选，**IBGP**对等体之间通告路由时不发生改变，也可以区域间防止环路

IBGP水平分割：就是实现区域内防环，当一个路由器从IBGP对等体学习到某条BGP路由，它不能再将该路由通告给任何IBGP对等体

由于水平分割的限制，路由传递受限，就需要用到全互联，但当规模大时，设备负担很大

路由黑洞问题，比如：**BGP**通告的路由，只靠**OSPF**无法得知，解决方法包括：使用**BGP**同步规则（路由），全互联，**MPLS**

路由通告规则：从外得到的消息对所有说，从内得到的消息对内有限说，只通告最优的一条BGP路由

报文类型：

Open: 建立TCP连接后，双方交换参数信息

Update: 向对等体通告路由信息

Keepalive: 了解对等体存活情况，该报文时间间隔为1/3的保活计时器时间（还用于确认Open报文）

Notification: 检测到错误时，告知对等体

路由发布到BGP（BGP不支持自动发现路由）

network: 将路由信息发布到BGP中，但一次只能发布一条路由

import-route: 路由引入到BGP中，一次性引入的是多条

aggregate: 向BGP发布汇总路由

BGP路径属性：

Pre_Value: 值越大路由越优先（华为私有），本地有效

Local_Preference: 只能在IBGP对等体之间传递，值越大越优先，缺省100，也就是保证AS内部最优路由

AS_Path: 个数越少路由越优先，通告给EBGP的时候，将本地AS号插入到原有AS_Path之前

Origin: 描述BGP路由的来源，三种类型-IBGP（最优），EBGP，Incomplete，适用于达到同一个目的网段存在多条BGP路由，而且其他条件相同

MED: 缺省只有当路由来自同一个相邻的AS时，BGP才会进行MED属性的比较，MED属性值越小路由越优先（内部有效）

Next_Hop: 记住一种重要情况-EBGP之间传递会将该属性值更新为传出AS的路由器的源IP地址，而且

Next_Hop也必须具体路由可达

Aggregator: 解决路由由汇总时造成明细路由的路径属性丢失，比如AS_PATH

Community: 将路由标记进行分类，比如划分不同的业务

BGP实验难度高内容多，暂时不看，理解原理就好

路由反射器，用于解决AS内部BGP路由传递问题的技术

路由反射器（RR）与所有的客户建立IBGP对等体关系，客户之间无需建立IBGP对等体关系，默认情况：

1. 路由反射器从自己的非客户对等体学习到一条IBGP路由，将该路由反射给所有客户
2. 路由反射器从自己的客户学习到一条IBGP路由，反射给所有非客户，以及除该客户之外的其他客户
3. 路由反射器只将自己使用的、最优的BGP路由进行反射

为防环，出现了水平分割，为解决水平分割的副作用，出现了路由反射器，但它又会带来防环这个问题

Originator_ID: 相当于个体的AS_path，标记，受到跟自己相同的标记的就丢弃

Cluster_List: 针对反射族，跟AS_path非常像

联邦，也是一个解决IBGP路由传递问题的做法

就像在一个大的AS内创建若干个小的AS，联邦AS内部的成员AS对于联邦外部并不可见，重邮、重师对外同属于重庆市

BGP路由优选原则：

任何一台BGP路由在参与优选之前都必须先经过检查。先检查BGP路由的Next_Hop是否可达，不可达则不可用

口诀：漂亮老（L A O）男人（M E N）

第五、和OSPF的区别

1. **定位与作用域**：OSPF是一个内部网关协议，专注于在单一自治系统内部高效、准确地计算最优路径，实现域内路由。BGP则是外部网关协议，核心任务是在不同自治系统之间传递路由信息并实施路由策略，控制流量如何进出AS以及如何在互联网骨干中传播。
2. **路径选择原则**：OSPF基于链路状态算法，使用带宽、延迟等综合度量值计算到达目的网络的最短内部路径，目标是效率。BGP基于路径矢量算法，其最佳路径选择首要考虑的是策略（如AS_PATH长度、本地偏好、MED值、运营商协定等），而不仅仅是技术指标，核心是控制与策略。
3. **收敛性与设计哲学**：OSPF设计追求快速收敛，通过触发更新和SPF算法快速响应网络变化，确保域内高效连通。BGP设计更强调稳定性、可扩展性和策略控制，更新是增量和触发式的，收敛相对较慢，但能支撑超大规模互联网路由表，并允许管理员精细控制路由的传播和选择。
4. **适用场景**：因此，OSPF是构建企业网、数据中心或运营商AS内部网络架构的基石；而BGP是连接不同企业、运营商乃至整个互联网的‘粘合剂’，是实现跨域互联和策略路由的全球标准协议。两者通常协同工作（OSPF管内部，BGP管外部）。

二、OSPF详解

第一、三张表

邻居表：hello报文对接后，就把对方加入进自己的邻居表里面，邻居的 **Router ID**（身份证号）、邻居所在的 **IP 地址**、和邻居相连的**本地接口**，以及当前状态

拓扑表：当邻居关系变成full之后，双方就会疯狂交换LSA，并且将收到的LSA信息放入LSDB--链路状态数据库，里面记录着哪台路由器连着哪台路由器、每个接口的带宽（Cost）是多少、哪个接口连着哪个网段。而且一个OSPF区域内所有路由器的LSDB是完全一致的

OSPF路由表：路由器会以自己为根节点，使用SPF算法，计算出来的结果，就是一张极其清爽的表格：目标网段、下一跳 IP、出接口、总开销 (Cost)。这就是 OSPF 路由表。张表是 OSPF **自己内部**的路由表（`display ospf routing`）。OSPF 把最优路径算出来后，还要把它们“上报”给路由器的全局路由表。如果此时全局路由表里有一条同样目的地的静态路由，而且静态路由优先级更高，那么 OSPF 辛辛苦苦算出来的这条路，就只能乖乖待在 OSPF 自己的表里，进不了全局地图，更进不了最终的转发表，当然，默认情况下，OSPF的优先级更高

第二、cost-开销

当你从 A 点去往 D 点，中间经过了 B 和 C。OSPF 怎么算总开销呢？

路由的累计 Cost，等于从源头到目的地，沿途所有出接口 (Outgoing Interface) 的 Cost 值之和，也就是，A->D和D->A的开销，是不一定相同的，甚至往返都是不同路径，它的计算是单向的（字节面试题）

第三、报文类型

hello报文，发现邻居、建立邻居关系、并且作为“心跳包 (Keepalive)”维持邻居关系，路由器一通电，就会向多播地址 `224.0.0.5` 疯狂发送 Hello 包，主要用于从 Down --> Init --> 2-Way 状态的推进

DD报文，它是LSDB的目录摘要，里面不含具体的路由细节，用于 ExStart（选主从）和 Exchange（互换目录）状态

LSR报文，看完目录后，路由器发现对方手里有几张自己没有的新图纸，用于向邻居精准索要缺失的 LSA 详情，进入 Loading 状态

LSU报文，面装着完完整整的 LSA（链路状态通告）的所有详细内容，有两种情况会发送该报文，1. 当收到邻居的 LSR 请求时，用 LSU 把详细的 LSA 发给对方。2. 当网络拓扑发生变化时（比如某根网线断了），路由器不需要等对方请求，会立刻主动触发一个 LSU 报文，把变化情况泛洪给全网

LSAck报文，OSPF是运行于IP层的，没有TCP帮忙做确认，所以以此来保证LSA的可靠传输，当收到对方发来的 LSU 时，必须马上回一个 LSAck 报文，告诉对方无需再重传

第四、网络类型

broadcast-广播多路访问

这是现网最最最常见的类型，支持广播，支持一对多，只要物理接口是以太网，OSPF 默认就会把它识别为 Broadcast 类型。为了减轻各设备的压力，OSPF在网络中会进行DR和BDR的选举，所有非DR、BDR路由器仅与DR和BDR建立邻接关系

P2P-点对点

通常出现在传统的串行链路（Serial 接口）、PPP 链路，或者两台路由器通过光纤直连的场景，无需选举，直接建立FULL邻接关系

NBMA非广播多路访问

必须手动配邻居：因为不能发组播 Hello 包，路由器没法自动发现对方，网络工程师必须敲命令 `peer x.x.x.x` 纯手工指定邻居，依然需要选举 DR/BDR

这里分享一个现代网络架构（尤其是你感兴趣的复杂数据中心路由和底层网络）中最经典的“魔改”操作：

在现代数据中心的 Leaf-Spine 架构中，两台交换机之间虽然用的是万兆以太网光纤直连，OSPF 默认会把它们识别为 **Broadcast（广播类型）**。

但是，既然是两台设备直连，这辈子都不可能第三台设备插进来，如果还按照 Broadcast 类型去傻等 40 秒（Wait Timer）来选举 DR 和 BDR，简直是极大的资源浪费和收敛延迟！

资深网络工程师的基操：

直接跑到这个以太网接口下，敲下一行命令：`ospf network-type p2p`。

这行代码直接欺骗 OSPF：“别管物理线是什么，给我当成**点到点（P2P）**来跑！”

- **收益瞬间拉满：**彻底跳过 DR/BDR 选举，邻居建立速度瞬间提升，不仅节省了 CPU 资源，还去除了不必要的 2 类 LSA（网络 LSA），让 LSDB 数据库变得极其清爽，路由计算速度大幅加快。

第五、各类LSA详解

第一类LSA

每一台运行OSPF的路由器都会阐述，描述了路由器的直连接口状况和接口Cost，其中也包含一些特殊的比特位，指示该路由器的特殊角色，还有接口的链路类型等等，经过第一类LSA的泛洪，区域内的路由器已经能够大致描述出本区域的网络拓扑（汇总信息）

第二类LSA

只有DR可以产生，DR收集了大家的信息，并绘制了一份详细完整的网络拓扑发给各个设备，里面包含各网段信息等等，解决了区域内的路由计算问题（同步信息）

第三类LSA

只有ABR区域边界路由器才能产生，主要是解决区域间的路由计算问题，包括路由信息和开销。

假设 Area 1 有个网段。ABR1 把它打包成 3 类 LSA，扔进 Area 0。这个 3 类 LSA 只在 Area 0 里泛洪，当 Area 0 另一头的 ABR2 收到后，ABR2 会重新生成一份全新的 3 类 LSA，发件人换成了 ABR2 自己，并且加上了 ABR2 到 ABR1 的开销，再扔进 Area 2，路由信息像接力棒一样，传遍了整个 OSPF 网络，但复杂的底层拓扑结构被死死地锁在了各自的区域内

第四类LSA

由ABR产生，实际上是一条到达ASBR的主机路由，度量值字段填写的是该ABR自己到达ASBR的cost值

现在来看看当外部路由被引入 OSPF 域后，OSPF 如何实现路由计算。首先，不妨通过一个例子来帮助大家理解外部路由的引入过程，假设有这么一座城，它被划分成了多个行政区，这座城市与外界完全隔绝，被封闭了起来，外面的信息进不来，里头的消息也出不去。有一天，这座城市在某个行政区开启了一个城门，这个城门成了城内居民去往城外的一个出口。现在，关于城外各种新奇事物的描绘经过这座城门被城内各个行政区的千家万户知晓了，大家都知道原来外面还有这么多好玩儿的地方。现在大家想要出城旅游了，目的地自然是这些城外的景点，但是要出城，就必须从城门走，那么大家就需要知道城门的所在。与城门同属一个行政区的居民能直接了解到城门的具体位置，这是因为同一个行政区内，所有的信息都是公开透明的。然而与城门不在同一个行政区的居民可就不知道城门的位置了，不知道城门的位置，自然就无法经过城门到达外面的世界。因此行政区的枢纽会将城门的信息通告给其他行政区，使得与城门不在同一个区的居民能够知晓到达城门的路径，最终，整个城市的居民都能够出城旅游了。

第五类LSA

去往区域外部的路由信息，由ASBR产生，一台路由器收到第五类LSA时，需要检查引入该外部路由的ASBR是否可达，无视区域边界，传遍整个OSPF自治系统（特殊区域除外），3类LSA跨区时，ABR会把它重新生成（换发件人），但是5类LSA完全不同！它从产生那一刻起，一路绿灯冲破Area 1，穿过Area 0，再冲进Area 2

第七类LSA

stub区域，想象一下：公司总部（Area 0）性能强悍，扛得住。但是下面某个偏远分公司（Area 2）用的是几百块钱的老旧路由器，CPU和内存极其可怜。当海量的5类LSA像海啸一样冲进Area 2时，这台边缘路由器的LSDB瞬间就被撑爆，直接死机。通俗来说，就是考虑到该区域无法承受这么多5类LSA，但是该区域的设备又要通外网，于是ABR就阻挡第五类LSA的同时，又下发默认路由，**潜台词：“外面的世界太复杂（5类），你别看了。你只要把所有不知道怎么走的流量全扔给我（默认路由），我来帮你转交！”**

再说说NSSA区域，现在，我这个stub区域阻挡了外网1的LSA，但又接一个外网2，所以需要将外网2的信息引入到其他内部区域，但stub区域内又不允许有第五类LSA存在，于是就发明了第七类LSA，这是ASBR产生的，ABR引入时，因内部Area 0无法识别7，所以会进行7转5操作

另外，普通的NSSA不会自动下发默认路由，但是totally NSSA会自动化下发默认路由

三、了解大规模集群网络

了解过AI吗？

对于AI，我平时主要从日常工具和架构演进两个维度在保持关注和实践

比如在开发Python自动化运维脚本，或者编写复杂的Prometheus告警规则时，AI能帮我极大提升编写和排错的效率

其次，站在监控架构演进的角度，我非常关注AIOps。在我之前搭建Prometheus监控时，我发现传统的‘静态阈值告警’很容易产生误报和告警风暴。我了解到业界正在利用AI对时序数据库（TSDB）进行动态基线预测和日志智能聚类，这能够极大地提升故障根因定位的效率

1、叶脊结构（物理支撑）

传统三层架构设计初衷是为了处理南北流量，如果不同汇聚层下的两台服务器要想通信，数据包就会穿过很多层，对核心交换机压力很大，而且由于 STP 的存在，为了防环，很多冗余链路是被阻塞的，导致带宽利用率极低；

相反，在 GPT 等大模型训练中，由于各服务器之间的数据同步，迁移等等，会产生海量的东西流量，如果用传统三层架构，核心交换机会瞬间被东西向流量打爆，形成严重的拥塞瓶颈

spine 脊交换机：相当于核心节点，spine 之间绝对互不相连，它们只连接所有的 Leaf

Leaf 叶交换机：相当于接入节点，服务器直接连在 Leaf 上。Leaf 之间也绝对互不相连，每个 Leaf 都要与所有的 Spine 相连（全互联）

在 AI 集群中，哪怕有一台服务器的数据晚到了几毫秒，整个集群的显卡都要停下来等（木桶效应），leaf-spine 架构使得任意两台跨机架服务器之间的通信，也只经过 3 条，保证集群内节点间延迟的绝对一致性和可预期性。而且它的水平扩展能力很强，可以轻松支持大规模集群

2、eBGP 和 ECMP（指挥系统）

OSPF 要求网络中的每一台连路由器都掌握完整的网络拓扑图，在全互联架构下，任何一根光纤的抖动，都会产生 LSA，并向整个网络泛洪，在大规模集群中，会导致交换机 CPU 急速飙升，导致瘫痪。

eBGP 不会全网泛洪，只会跟直连的邻居交换路由信息，只会影响局部；而且 BGP 拥有完美的防环机制，可以解决复杂物理连线下的环路问题；最重要的是 BGP 拥有极其丰富的路径属性，使得工程师可以通过手段精准地控制业务流量走向，提高带宽利用率

ECMP 负责流量分发，基于流负载分担（哈希）：路由器根据 IP 报文的 **五元组** 信息（是指源 IP 地址，源端口，目的 IP 地址，目的端口，和传输层协议这五个量组成的一个集合。例如：192.168.1.1 10000 TCP 121.14.88.76 80 就构成了一个五元组）将数据分成不同的流。具有相同五元组信息的 IP 报文属于同一个流。转发数据时，路由器把不同的数据流根据算法从多个路径上依次发送出去。

说到这里，来分析一个数据网络的现实情况：

在双 11 的大促期间，北京机房的一台承载核心数据库的虚拟机（VM），因为底层物理机 CPU 快冒烟了，需要不停机紧急漂移到上海机房，要求：这台虚拟机在漂移的过程中，**IP 地址绝对不能变，MAC 地址也绝对不能变**，甚至连建立好的 TCP 连接都不能断开，很显然，传统网络是办不到的，因为在传统网络中，北京和上海之间隔着无数个三层路由器。三层路由器天生就是用来隔离二层广播域的。虚拟机一跨过三层路由器，IP 网段就必须变。如果用传统的 VLAN 把两地强行拉通，那无尽的“广播风暴”会瞬间让跨地域骨干网瘫痪

VXLAN 技术应运而生，当北京机房的 VM 发出一个原始的二层数据帧，源 MAC 是自己，目的 MAC 是上海机房的另一台 VM，这台 VM 以为它们就在同一个交换机下面，北京的 Leaf 交换机收到这个二层帧后，直接把它打包，在外面套上一个 VXLAN 头，再套上 UDP 头，最后套上外网的 IP 头（源 IP 是北京 Leaf，目的 IP 是上海 Leaf），底层网络根本无感知

VXLAN 虽好，但是早期的它是靠泛洪来学习 MAC 地址，在云环境里面也会是灾难，如果让上海的设备拿到北京设备的 MAC 地址呢？网络架构师们进行了一次降维打击级的创新：把高高在上的 BGP 协议拉下来，去接管底层的 MAC 地址。这就是 EVPN。现在，当一台虚拟机在北京机房上线时，北京的 Leaf 交换机会立刻学到它的 MAC 和 IP。然后，北京 Leaf 会生成一条 EVPN Type-2 路由，通过 BGP Update 报文发送给全网

1. **准备漂移：**虚拟机的数据和内存状态通过底层存储网络，从北京物理机同步到了上海物理机。
2. **上海“苏醒”（触发 GARP）：**虚拟机在上海机房“苏醒”的瞬间，会本能地向外发一个免费 ARP (GARP) 报文，宣告自己的存在。
3. **上海 Leaf 捕获并宣告：**上海的 Leaf 交换机收到了这个报文，发现这个 MAC 和 IP 突然出现在了这里。它立刻生成一条全新的 **EVPN Type-2 路由**！
4. **核心机制：MAC 移动性序列号 (MAC Mobility Sequence Number)：**
BGP 怎么知道谁是旧的，谁是新的？上海的 Leaf 在发送这条路由时，会利用 BGP 的扩展团体属性打上一个“序列号”。北京当初发的是 `Seq=0`，上海这次发的就是 `Seq=1`。
5. **全网路由刷新：**
这条 `Seq=1` 的 BGP 路由瞬间传遍全网（可能通过我们上一个话题聊到的 RR 路由反射器）。其他机房的交换机收到后，一看：“哟，序列号变大了，说明这个 VM 搬家了！”它们立刻更新自己的转发表，把指向该 VM 的隧道终点从北京改成了上海。
6. **北京 Leaf 的“放手”：**
北京的 Leaf 交换机也收到了这条 `Seq=1` 的路由，它明白这个 VM 已经不在自己这里了，于是默默地撤销本地的转发表项。

整个过程在毫秒级完成，业务对底层的跨地域迁移毫无感知。TCP 没断，IP 没变，MAC 没变。

3、无损网络与 RoCEv2（提速）

在AI大模型训练时，海量的内存拷贝和内核态/用户态的频繁切换，不仅极大消耗 CPU 资源，还会带来毫秒级的延迟。这在按微秒计较的 AI 训练中是不可接受的，TCP 遇到拥塞默认会丢包，然后触发重传机制（比如拥塞窗口减半）。在 AI 训练中，所有 GPU 都在同步等数据，哪怕只丢了一个包，整个集群成百上千张 GPU 都要停下来等这一个包重传，这被称为灾难性的“长尾效应”

RDMA可以解决这个问题，它运行数据直接从网卡写到目标服务器的内存，完全绕过操作系统的内核（个人感觉有点像零拷贝）。RDMA 性能虽高，一旦网络丢包，它的性能会呈现断崖式下跌，甚至不如 TCP，这时候就需要PFC和ECN

PFC时基于优先级的流量控制，简单粗暴地理解：如果交换机 A 发现自己某个高优先级队列的内存缓存（Buffer）快被打满了，它会向上一跳的交换机 B 紧急发送一个“PFC 暂停帧”。B 收到后会立刻停止发包，以此类推，拥塞的压力会一层层向后传递，直到源头的网卡。

ENC是显式拥塞通知，交换机发现队列水位越过某个警戒线（快要拥塞但还没满）时，它不会丢包，也不会发 PFC，而是在数据包的 IP 头里打上一个“拥塞标记（CE）”。接收端服务器收到带有标记的包后，会通过特殊的控制报文反向通知发送端：“前面路况开始拥挤了，你稍微把发送速率降一降”

4、Telemetry（运维与观测）

用SNMP或者python脚本去拉取设备的各项指标，拉取精度太低，不够准确，因为AI训练的流量波动很大很快，可能无法看见一瞬间的拥塞

telemetry用push模式代替pull模式，交换机会通过 **gRPC 等高效协议**，主动、持续地向外的时序数据库推送极其精细的运行状态，带内网络遥测是目前最前沿的技术之一。探针数据直接封装在业务数据包的头部里。数据包每经过一台交换机，交换机就会把自己的当前处理延迟、队列深度加进去，数据包到达后再由后端分析，实现秒级感知，并通过智能修改相关策略调整路径流量，实现网络自愈

